The order module: how to apply and use it

announce-reactive, branch feat/order-reactive, commit f6e3f7bb, 2026-09-26. Companion to doc/order-context.md (design and deviations) and doc/cqrs-reactive.md (framework).

1. What you get

The Maven module **order** replaces the legacy litemall order module. It is one Spring Boot service on the cqrs-reactive framework (OpenCQRS on EventSourcingDB, R2DBC MySQL read models, Spring Cloud Stream on Kafka) that hosts four bounded contexts. Each context has its own aggregate, projections, jobs, REST endpoints and public topic, and shares nothing with the others but the public events of the **order-api** module. An ArchUnit test enforces that.

Context (package root)	Aggregate / subject	Read tables	Publishes	Consumes
auction org.linlinjava.announce.auction	Auction /auction/{announceId}	announce_auction, announce_bid, announce_auction_event, announce_auction_seller	auction.v1	announce.v1, payment.v1
order org.linlinjava.announce.order	Order /order/{orderId}	announce_order, announce_order_snapshot, announce_order_seller	order.v1	announce.v1, auction.v1, messaging.v1, payment.v1, shipping.v1
payment org.linlinjava.announce.payment	Payment /payment/{orderId} SellerAccount /seller-account/{userId}	announce_payment, announce_seller_account	payment.v1	order.v1
shipping org.linlinjava.announce.shipping	Shipment /shipment/{orderId}	announce_shipment	shipping.v1	order.v1

Money flows through a marketplace payment provider, never through the platform: the buyer pays the provider, the seller ships through a carrier, and the payout to the seller is released only after the carrier confirmed delivery (or dispatch, by configuration) and a dispute window elapsed. Provider and carrier facts arrive as signed webhooks; a click by a member never sets PAID, SHIPPED or DELIVERED.

Three doors open a deal: an auction that closes with a winner, a Buy click on a fixed-price announce that the seller accepts, or a chat offer accepted in the messaging module. From there every deal follows the same path: AGREED, PAYMENT_PENDING, PAID, SHIPPED, DELIVERED, RELEASING, SETTLED, with CANCELLED before payment and DISPUTED, REFUNDING, REFUNDED after it.

2. Apply the database schema

The read models live in **db-reactive/sql/announce_order.sql**, which runs after announce_table.sql and announce_auction_chat.sql (the bootstrap file announce_reactive_db.sql already contains all three, regenerate it with sql/build_announce_reactive_db.sh). Applying it to an existing database drops and recreates the order module's tables and removes the litemall tables announce_order_goods and announce_cart. On the live announce_reactive_db all of them were empty on 2026-09-26. Run once, on the MySQL server of the environment:

```
cd db-reactive/sql
{ echo "DROP TABLE IF EXISTS announce_order_goods; DROP TABLE IF EXISTS announce_cart;"; cat announce_order.sql; } \
  | mysql -uannounce -p announce_reactive_db
```

For a fresh database use the bootstrap instead: `mysql -uroot -p < db-reactive/sql/announce_reactive_db.sql` followed by the seed `announce_data_en.sql`. The two framework tables (cqrs_command_log, cqrs_readside_progress) are created by the service itself at start-up.

To validate the DDL without touching a real server, load the three files into a throwaway container:

```
docker run -d --name announce-order-ddl -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=announce_reactive_db \
```

```
  -e MYSQL_USER=announce -e MYSQL_PASSWORD=announce -p 33070:3306 mysql:8.4
for f in announce_table.sql announce_auction_chat.sql announce_order.sql announce_data_en.sql; do
  docker exec -i announce-order-ddl mysql -uroot -proot announce_reactive_db < db-reactive/sql/$f; done
```

When a table changes, regenerate its reactive mapper from inside db-reactive (the generator resolves its target directories relative to the working directory) and reactify it:

```
cd db-reactive
mvn -P mbg mybatis-generator:generate -Dmybatis.generator.tableNames=announce_order \
    "-Dmbg.url=jdbc:mysql://127.0.0.1:3306/announce_reactive_db?useSSL=false&allowPublicKeyRetrieval=true" \
    -Dmbg.user=announce -Dmbg.password=...
python3 mybatis-generator/reactify_mapper.py . AnnounceOrder
```

3. Build and run

Prerequisites: Java 21 for Maven (the java on PATH is 11; use JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64), Docker, the local stack `docker compose -f docker/cqrs/docker-compose.yml up -d` (EventSourcingDB on 3000, Kafka on 9092; topics are auto-created) and the native MySQL with the schema of section 2.

```
mvn -N install                                  # once after a parent pom change
mvn -pl db-reactive,cqrs-reactive,cqrs-reactive-test,announce-management-api,order-api \
    -Dmaven.test.skip=true install
mvn -pl order -DskipTests package
java -jar order/target/order-0.0.1-SNAPSHOT-exec.jar     # or: mvn -pl order spring-boot:run
```

The service listens on port 8083 and registers with Eureka when one is running. Sibling worktree sessions overwrite the shared db-reactive jar in ~/.m2, so when in doubt build with the reactor: `mvn -pl order -am -DskipTests package`.

Configuration (order/src/main/resources/config/application.yml; every value can be overridden with the usual Spring property or environment variable):

Key	Default	Meaning
spring.r2dbc.mybatis.r2dbc-url / username / password	localhost:3306/announce_reactive_db, announce	Read models and framework tables (env ANNOUNCE_R2DBC_URL, ANNOUNCE_DB_USER, ANNOUNCE_DB_PASSWORD)
esdb.server.uri / api-token	http://localhost:3000 / secret	EventSourcingDB (env ANNOUNCE_ESDB_URI, ANNOUNCE_ESDB_TOKEN)
spring.cloud.stream.kafka.binder.brokers	localhost:9092	Kafka (env ANNOUNCE_KAFKA_BROKERS)
order.currency	DKK	Currency of deals created without one
order.window.pay	48h (12h to 7d)	After AGREED; expiry cancels the order
order.window.ship	7d (3d to 7d)	After PAID; expiry refunds the buyer
order.release.on	DELIVERY	DELIVERY or DISPATCH: what the payout waits for
order.release.delivery-grace	3d (0 to 14d)	Dispute window after delivery (after dispatch in DISPATCH mode); ConfirmReceipt ends it early
order.release.dispatch-grace	14d (7d to 30d)	Release after dispatch when the carrier never reports delivery
order.fee.default.percent / fixed-cents	8 / 500	Platform fee, fixed at agreement; order.fee.categories.<id>.* per category
order.seller-account.required, auction.seller-account.required	true	AcceptPurchase / OpenAuction refused until the seller's payout account is enabled
auction.default-increment-cents / default-auto-extend-sec / default-duration	1000 / 120 / 7d	Settings of an auction opened from a published AUCTION announce
payment.provider, payment.fake.secret	fake, fake-webhook-secret	fake or stripe (stub); HMAC key of the fake webhook
shipping.carrier, shipping.fake.secret	fake, fake-carrier-secret	fake or postnord (stub); HMAC key of the fake webhook

Key	Default	Meaning
order.jobs.*, auction.jobs.*, shipping.jobs.*	enabled, every minute (tracking every 5 min)	The timers; disable in tests and drive them by hand

Windows outside their allowed range fail the start-up on purpose.

4. Topics

Every record is a class of order-api (or announce-management-api) annotated with @PublicEvent(type, version); Kafka headers carry event-type, event-version, event-id, subject, correlation-id. Consumers de-duplicate on the event id (the module uses it as command id, so a redelivered record replays the stored result).

Topic	Producer	Records	Consumers
announce.v1	announce-management	AnnouncePublishedV1 (snapshot with priceCents, currency, saleType, categoryId, expiresAt), AnnounceEditedV1, AnnounceWithdrawnV1, AnnounceExpiredV1, AnnounceSoldV1	auction (open, cancel), order (snapshot)
auction.v1	order / auction	AuctionOpenedV1, BidPlacedV1, AuctionClosedWonV1, AuctionClosedUnsoldV1, AuctionSnaggedV1, AuctionCancelledV1	order (the auction door)
order.v1	order / order	OrderAgreedV1, OrderPaidV1, OrderShippedV1, OrderDeliveredV1, OrderSettledV1, OrderCancelledV1, OrderRefundedV1, PayoutReleaseRequestedV1, RefundRequestedV1	announce-management (MarkSold, RelistAnnounce), payment, shipping, messaging
payment.v1	order / payment	PaymentIntentCreatedV1, PaymentConfirmedV1, PaymentFailedV1, PayoutSentV1, RefundSentV1, SellerAccountUpdatedV1	order, auction
shipping.v1	order / shipping	ShipmentRequestedV1, ShipmentDispatchedV1, ShipmentInTransitV1, ShipmentDeliveredV1, ShipmentFailedV1	order
messaging.v1	messaging (stub in order-api, package org.linlinjava.announce.messaging.api, type conversation.offer-accepted)	OfferAcceptedV1(conversationId, announceId, buyerId, sellerId, amountCents, currency, acceptedAt)	order (the offer door)

5. REST API

Authentication happens at the gateway in this phase: members arrive as the header **X-User-Id**, staff as **X-Admin-Id**. An optional **X-Command-Id** makes a write idempotent (a retry with the same id replays the stored answer; for a bid it also fixes the bid id). Writes answer with the aggregate state the command produced; reads come from the read model a moment later. Errors are RFC 7807 problem details with a stable `code` (409 for a rule, 403 for ownership, 404 for an unknown subject).

Method and path	Who	Does
GET /api/auctions/{announceId}	anyone	Auction row and its bids
POST /api/auctions/{announceId}/bids {amountCents, maxAmountCents?}	member	Places a bid: not the seller, at least start price or current + increment, inside the window; extends the end inside the anti-sniping window
POST /api/auctions/{announceId}/snag	member	Buys at the snag-it price while no bid exists
POST /api/auctions/{announceId}/cancel {reason}	seller	Cancels before the first bid
DELETE /api/auctions/{announceId}/bids/{bidId}	bidder	Retracts a bid; the next highest becomes current
POST /api/orders {announceId}	buyer	Fixed-price door: proposes a purchase of a published, non-auction goods announce
POST /api/orders/{id}/accept | decline {reason}	seller	Accept fixes the fee and the payout; refused until the seller's payout account is enabled
POST /api/orders/{id}/cancel {reason}	buyer or seller	Before payment only
POST /api/orders/{id}/confirm-receipt	buyer	Ends the dispute window early and releases the payout
POST /api/orders/{id}/dispute {reason}	buyer	After payment; blocks the payout until staff resolve
GET /api/orders/{id}, GET /api/orders?role=buyer|seller&page;=&size;=	party	Read model
GET /api/orders/{id}/payment	party	Payment state incl. the client secret the buyer needs to pay
POST /api/orders/{id}/shipment {service, parcelSize, fromAddress, toAddress}	seller, after PAID	Books with the carrier; answers label URL and tracking number
GET /api/orders/{id}/shipment	party	Tracking state
POST /api/sellers/me/account, GET /api/sellers/me/account	seller	Starts onboarding at the provider (the fake verifies at once) and reads the account
POST /api/payments/webhook/fake, POST /api/shipping/webhook/fake	provider / carrier	Signed webhooks (section 6); the stripe and postnord hooks answer 501 until the adapters land
POST /api/admin/orders/{id}/resolve-dispute {resolution: RELEASE|REFUND, note}	staff	Ends a dispute
POST /api/admin/orders/{id}/release	staff	Runs the release rule for one order (409 dispute-open, not-delivered)
GET /api/admin/orders/{id}	staff	Any order
POST /api/admin/bids/{bidId}/cancel {reason}, POST /api/admin/auctions/{announceId}/cancel {reason}	staff	Removes a bid or an auction
POST /api/admin/auctions/close-due	staff	Runs the closing job once

6. Walkthrough: a fixed-price deal with the fakes

With the local stack, the service on 8083 and announce-management publishing on announce.v1 (an announce of kind GOODS with sale type FIXED_PRICE published by member 7). Buyer is member 9. The fake provider and carrier need signed webhooks: the signature is the hex HMAC-SHA256 of the raw body with the secret from the configuration.

```
H='-H Content-Type:application/json'
# 0. the seller needs a payout account once (the fake provider verifies immediately)
curl -s -X POST localhost:8083/api/sellers/me/account -H X-User-Id:7
```

```
# 1. the buyer asks, the seller accepts
ORDER=$(curl -s -X POST localhost:8083/api/orders -H X-User-Id:9 $H \
        -d '{"announceId":"<announce id>"}' | jq -r .id)
curl -s -X POST localhost:8083/api/orders/$ORDER/accept -H X-User-Id:7          # AGREED, fee and payout fixed
curl -s localhost:8083/api/orders/$ORDER/payment -H X-User-Id:9                  # PAYMENT_PENDING, clientSecret
# 2. the provider confirms the payment (webhook)
sign() { printf '%s' "$1" | openssl dgst -sha256 -hmac "$2" | awk '{print $2}'; }
B='{"providerEventId":"evt-1","kind":"CONFIRMED","orderId":"'$ORDER'"}'
curl -s -X POST localhost:8083/api/payments/webhook/fake $H \
     -H "X-Fake-Signature: $(sign "$B" fake-webhook-secret)" -d "$B"
# 3. the seller books the shipment, the carrier reports dispatch and delivery (webhooks)
curl -s -X POST localhost:8083/api/orders/$ORDER/shipment -H X-User-Id:7 $H \
     -d '{"service":"standard","parcelSize":"M"}'
for K in DISPATCHED DELIVERED; do
  B='{"carrierEventId":"c-'$K'","kind":"'$K'","orderId":"'$ORDER'"}'
  curl -s -X POST localhost:8083/api/shipping/webhook/fake $H \
       -H "X-Fake-Signature: $(sign "$B" fake-carrier-secret)" -d "$B"
done
# 4. the buyer confirms receipt (or the release job runs after order.release.delivery-grace)
curl -s -X POST localhost:8083/api/orders/$ORDER/confirm-receipt -H X-User-Id:9   # RELEASING
# 5. the provider reports the payout (webhook) -> SETTLED, OrderSettledV1 on order.v1
B='{"providerEventId":"evt-2","kind":"PAYOUT_SENT","orderId":"'$ORDER'","ref":"tr_1"}'
curl -s -X POST localhost:8083/api/payments/webhook/fake $H \
     -H "X-Fake-Signature: $(sign "$B" fake-webhook-secret)" -d "$B"
```

Fake payment webhook kinds: CONFIRMED, FAILED, PAYOUT_SENT, REFUND_SENT (payments, field orderId) and ACCOUNT_UPDATED (seller accounts, fields userId, status, payoutsEnabled). Fake carrier kinds: DISPATCHED, IN_TRANSIT, DELIVERED, FAILED (optional at, reason). A providerEventId or carrierEventId already applied is acknowledged with 204 and ignored; a bad signature is 401.

An auction instead of a Buy click

An announce published with sale type AUCTION opens an auction automatically (start price = the announce price, increment, anti-sniping window and duration from auction.*, capped by the announce expiry). Members bid; when the end passes, AuctionClosingJob (or POST /api/admin/auctions/close-due) closes it, AuctionClosedWonV1 goes out and the order context creates the deal AGREED with the winner at the winning bid. From there the walkthrough continues at step 2. A chat offer accepted in the messaging module (OfferAcceptedV1) creates the deal the same way at the offer amount.

7. Timers and what they do

Job	Reads	Sends	Effect
AuctionClosingJob	announce_auction OPEN with end_time passed	CloseAuction	CLOSED_WON (highest bid meets the reserve) or CLOSED_UNSOLD
OrderPayWindowJob	announce_order AGREED / PAYMENT_PENDING older than order.window.pay	Cancel (SYSTEM)	CANCELLED, OrderCancelledV1; the announce stays PUBLISHED
OrderShipWindowJob	announce_order PAID older than order.window.ship	ExpireShipWindow	REFUNDING, RefundRequestedV1; after the provider's refund REFUNDED, OrderRefundedV1; announce-management relists the announce
OrderReleaseJob	announce_order DELIVERED or SHIPPED	ReleasePayout	PayoutReleaseRequestedV1 when the release rule passes; a no-op while the grace runs
ShipmentTrackingJob	announce_shipment REQUESTED / DISPATCHED / IN_TRANSIT	RecordCarrierEvent per polled event	Safety net for carriers that do not push webhooks

Every job has a public runOnce() and is a pass over a read model that sends commands; the aggregate decides. There is no scheduler state in the event store.

The release rule

The payout is released when the payment was confirmed, no dispute is open, and: in DELIVERY mode the carrier reported delivery and the delivery grace elapsed (or the buyer confirmed receipt), or the parcel was dispatched and the dispatch grace elapsed without any delivery event; in DISPATCH mode the parcel was dispatched and the delivery grace elapsed since dispatch (or the buyer confirmed). A dispute answers 409 dispute-open, a shipped-but-not-delivered order in DELIVERY mode 409 not-delivered. Staff resolve a dispute with RELEASE (payout at once) or REFUND.

8. Tests

```
mvn -pl order test                                    # aggregate tests, ArchUnit boundaries, end-to-end on container
mvn -pl announce-management -am -P '!webapp' test     # the announce side (SOLD on OrderPaidV1, relist on refund)
mvn -pl db-reactive test        # needs a MySQL with the schema: ANNOUNCE_R2DBC_URL=r2dbc:mysql://host:port/db
```

The end-to-end test needs Docker only (images thenativeweb/eventsourcingdb:1.2.0, apache/kafka-native:3.9.1, mysql:8.4). It stands in for announce-management and messaging with PublicEventTestProducer from the test kit, probes order.v1, moves time with a mutable Clock bean and runs the job passes by hand (the jobs are disabled in the test profile). Scenarios: auction to settlement, fixed price with early receipt confirmation, chat offer, dispute guard, ship-window refund, duplicate webhooks, seller bid and below-increment bid rejected.

9. Known limits and follow-ups

Stripe Connect and PostNord are stubs behind the PaymentProvider and Carrier ports (payment.provider=stripe or shipping.carrier=postnord makes every call fail on purpose). Proxy bidding is stored (maxAmount) but not executed. An auction opened from an announce has no reserve or snag-it price until the announce carries those fields. OfferAcceptedV1 is a stub in order-api: the messaging module must publish exactly the type conversation.offer-accepted with those field names, or move the record into messaging-api. The seller gate refuses AcceptPurchase and OpenAuction until payment.v1 delivered SellerAccountUpdatedV1 with payouts enabled; an AUCTION announce published before that is logged and skipped, not reopened later. Authentication is still the gateway's job (X-User-Id, X-Admin-Id).